UNIT-3

1. Generate Text Embedding and Perform Semantic Similarity Search

Aim

To generate numerical text embeddings using a pre-trained Sentence Transformer model and perform semantic similarity search between sentences.

Algorithm

1. Import the required libraries for text processing and embedding generation.
2. Define a collection of sample sentences related to different topics.
3. Load the pre-trained Sentence Transformer model.
4. Convert the sentences into numerical vector embeddings.
5. Accept a query sentence from the user.
6. Generate the embedding for the query sentence.
7. Calculate cosine similarity between the query embedding and all sentence embeddings.
8. Display the sentences ranked according to their similarity scores.

Program:

```
# Install required library

!pip install -q sentence-transformers


# Import libraries

from sentence_transformers import SentenceTransformer

from sklearn.metrics.pairwise import cosine_similarity


# Sample documents

sentences = [

    "Artificial Intelligence enables machines to learn.",

    "Machine Learning is a branch of Artificial Intelligence.",

    "Python is a popular programming language.",

    "Deep Learning uses neural networks.",

    "Natural Language Processing deals with human language."
```

```
]

# Load embedding model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Generate embeddings
embeddings = model.encode(sentences)

# Get user query
query = input("Enter your search query: ")

# Generate query embedding
query_embedding = model.encode([query])

# Calculate cosine similarity
similarity_scores = cosine_similarity(
    query_embedding, embeddings
)[0]

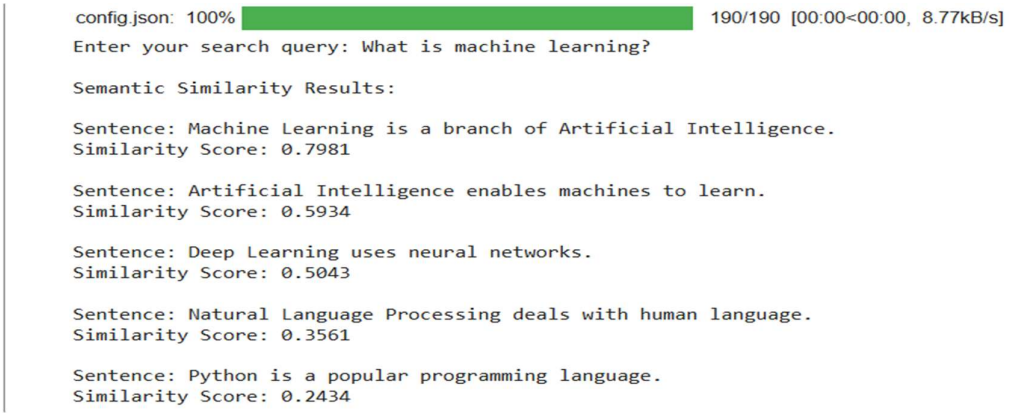
# Display results
print("\nSemantic Similarity Results:")

ranked_results = sorted(
    enumerate(similarity_scores),
    key=lambda x: x[1],
    reverse=True
)

for index, score in ranked_results:
```

```
print(f"\nSentence: {sentences[index]}")
print(f"Similarity Score: {score:.4f}")
```

output:



Result:

Thus, text embeddings were successfully generated using the Sentence Transformer model, and semantic similarity search was performed using cosine similarity.